

结束语 | 未来让Rust带你“锈”到起飞

唐刚 · Rust 语言从入门到实战



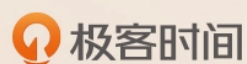
唐刚

Rust 语言中文社区联合创始人

你好，我是 Mike。

恭喜你学完了这门课程，也刷新了自己对 Rust 世界的认知。未来我希望你以一种轻松愉快的姿态去迎接 Rust，大胆地使用 Rust 编程，发挥你的想象力，因为 Rust 能带你到达你想象中的任何地方。

所以不要犹豫，加入进来吧！

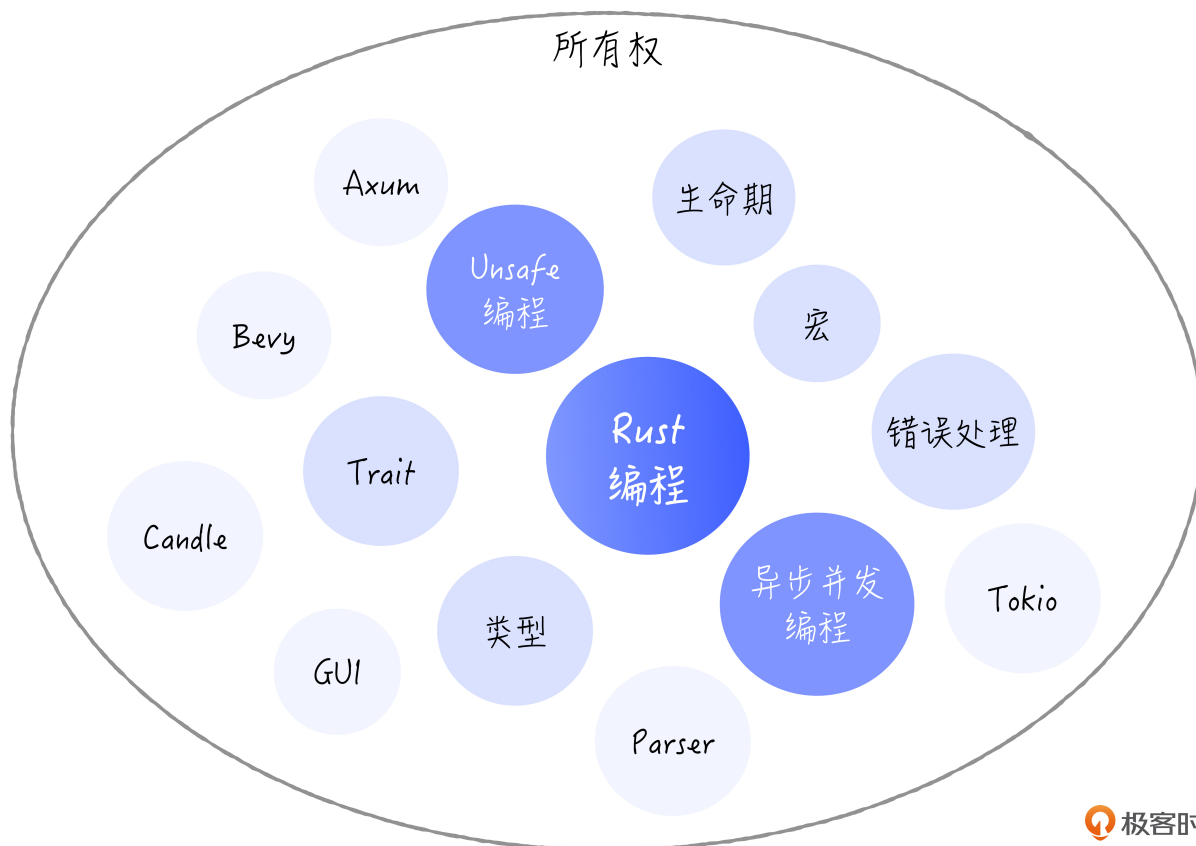


你好，我是你的老朋友 Mike。

30 讲的课程很快就结束了，感谢你一路的支持与陪伴，每一次评论区的互动都能让我感觉到写这门课的重要意义。我看到了你在不断思考中对 Rust 的理解越来越深，畏难情绪也随之一分分减少，很长一段时间我一起床就会打开评论，看看你有哪些新思考、新困惑，这也成了我每天的动力来源。

看到越来越多的朋友加入学习，逐渐“锈”化，我想我写这门课程最基本的目标达到了——让你以比较轻松的心态去掌握 Rust 的基础知识，为你以后用 Rust 去解决生产上的难题打下坚实的基础。

在这几个月里我们了解了 Rust 很多非常重要的特性，对所有权、Trait、类型还有异步编程和 Unsafe 编程等重要概念有了更深的理解，此外还转变了对 Rust 小助手的态度，真正把它当成了我们的好伙伴。这都是我们共同努力的结果。



而这最后一节课也同样珍贵，所以我还是想再“唠叨唠叨”，带你回顾我们课程里最重要的一些东西。希望你不仅能够掌握基本的 Rust 代码应该怎么写，还能理解 Rust 为什么要这样设计。

Rust 所有权是怎么来的？

要说 Rust 里最重要的东西，一定少不了所有权，都说重要的事情说三遍，在课程中我说了又何止三遍，所以我想用这最后一点点宝贵的时间再来讲一讲所有权从何而来，又何以至此的。我们都知道 Rust 成长于巨人的肩膀上，所以我们可以从 C 语言说起。

C 语言的手动内存管理

C 的做法是完全交给程序员自己来管理堆内存，也就是分配内存及释放内存。比如：

你 malloc 了一块内存，但是如果忘了写 free 的话，就会造成内存泄露。

如果你在不同的地方 free 了两次的话（指针的多次 free），就会造成未定义行为，比如段错误。

你定义了一个指针，但是没有初始化的话，对指针的解引用（未初始化指针访问）也会生成未定义行为。

你用一个指针指向了一个分配的堆内存，但是在某个地方被 free 掉了，但是你又对这个指针解引用的话（释放后访问），又会出现未定义行为。

.....

因此在 C 语言中，出现了一套避坑的最佳实践总结，其中最重要的一条原则就是**谁分配谁释放**，这是一条人为的约定。

看起来 C 语言里的内存管理完全是手动的，其实不尽然，手动管理的主要是堆内存部分。栈上内存的管理其实是自动的。因为栈上的内存管理只涉及到一个栈顶指针和计算偏移量，会以函数栈帧为单元压栈出栈，这块儿内存会随着函数层次的调用自动分配和回收。编译器保证了这块的内存不会发生错误的管理。

Java 的 GC

由于 C 语言内存管理非常痛苦，总是避免不了各种内存崩溃的问题。人们想出了另一种方案，用一个管家自动帮我们清理那些不要的“垃圾”，这正是 Java 等后面一系列语言的做法。这个管家就是 GC（垃圾回收器）。这种方案的思路就是，程序员只管分配内存，用完就不管了，GC 会自动跟踪这些资源，在适当的时候自动回收。这大大释放了程序员的心智负担，有人给你负责收尾了，多大的好事儿呀！

但是 GC 方案的代价也是明显的，比如不管使用什么跟踪回收算法，它始终存在 STW（Stop the World）的问题。它总是在程序员与底层之间隔离了一层，很难做到精确控制。这两点在做上层应用开发的时候问题不明显。但是在涉及到一些关键系统的时候，影响就很大了。大部分情况下，运行性能也会有影响。

C++ 的智能指针

C++ 是一门很牛的语言，它一方面做到了完全兼容 C。另一方面又在现代语言的方向上做了很多探索。🔗RAII（Resource acquisition is initialization）或者 SBRM（Scope-based Resource Management）就是在 C++ 里面提出的。

它的思路其实就基于上面提到的那个情况：**既然栈上的内存管理是自动的，那如果找到一种办法将堆上的资源与栈上的内存资源绑定，那不就可以自动管理堆上的内存资源了吗？**让堆上的内存资源随着栈上的资源一起创建一起回收，同时这也是使用固定尺寸的结构管理非固定尺寸的结构的方法。

既然是绑定，那就会存在一个指向的关系，也就是需要在栈上的一个固定尺寸的结构中有一个指向待管理的堆内存资源的指针，这个指针存的就是那个堆内存资源的内存地址。这个固定尺寸的结构就是**智能指针**。因此在 C++ 中你会看到 `unique_ptr`、`shared_ptr` 等智能指针。

Rust 的所有权

Rust 完全沿用了这个优秀的思想和方法，却没有从语法上兼容 C 的负担。Rust 进一步把这个负责管理资源的智能指针冠以**所有权**。对应的，Rust 中的原始指针 raw pointer 就没有所有权的概念，因为它并不参与负责释放的工作。

智能指针有很多种策略，与 C++ 版本对应的，Rust 的 `Box<T>` 就对应 `unique_ptr`，独占所有权，`Arc<T>` 就对应 `shared_ptr`，共享所有权。从前面的学习可以看到，`String`、`Vec` 其实也是智能指针，它们具有所有权属性。`Box` 就是泛化版本的 `String`、`Vec`，`String`、`Vec` 是特化版本的 `Box`。当然还存在其他智能指针，你甚至可以构思自定义策略，定义自己的智能指针。

当然所有权不仅涵盖智能指针和堆上的资源。栈上的资源也有所有权。比如 `i8`、固定尺寸的结构体等等，默认都放在栈上，它们也是具有所有权的。所有权概念的核心在于一个栈上变量（当然是固定尺寸的）是否负责管理资源。对于 `i8` 这样固定尺寸的结构体等类型来说，它们本身的值就存在栈上分配的那一小块内存空间中，因此会随着栈的回收自动被回收掉，因此也相当于这些变量也管理了资源。因此它们也是具有所有权的。

所有权之上：Move 与 Copy

而 Move 语义与 Copy 语义是另一码事，你可以把它理解成所有权之上的上层概念。在 Rust 中，只要某种类型实现了 `Copy trait`，那么在使用 `=` 号再赋值时，就会执行 Copy 语义。反之，会执行 Move 语义。

为了方便操作，Rust 对语言和标准库里很多固定尺寸的基础类型实现了 Copy 语义，而对同样是固定尺寸的结构体却默认没有实现 Copy 语义。**Move 语义就是移动所有权，防止过大的内存复制开销。Copy 语义就是复制所有权，创建新的所有权，原来那个所有权仍然被原来的变量持有。**

'a 生命期标注

Rust 通过所有权机制，实现了对资源的“自动”管理。但是这种管理方式也会带来一些新的负担，那就是对所有权变量的引用生命期的有效性分析就会比较复杂，因为需要确保这些引用不能超出所有权型变量的生命期。这块目前业界还无法做到逻辑级别的分析，只是从函数的签名定义上进行分析。**因此在必要的时候，Rust 编译器还需要程序员来提供一些信息，这就是 'a 生命期标注的来源。**

严格的引用生命期分析是 Rust 开辟的新领域，比 C++ 更进一步。所以说 Rust 不仅是工程上的创新，也在计算机语言理论上做了一些创新。

所有权有三态：**所有权变量、不可变引用、可变引用**。这三种形态几乎贯穿 Rust 语言的所有方面。`Box<Self>`、`Arc<Self>` 等实际只是所有权变量的变体。其实 Rust 的所有权就是这么一回事儿，就是这些内容了。

Rust 难学的点在哪儿？

除了所有权这一块之外，Rust 普遍被认为难学的还有它**严格的类型系统**，很多人不习惯。

1. 类型化与类型参数（泛型）各种形式的组合。
2. 类型嵌套的洋葱结构会让类型变得很长，一旦出现类型不匹配的问题，初学者容易被编译器给出的信息吓住。
3. 全新的 trait 理念。

Rust 中的类型确实比较严格，比如字符串，C 里就一个 `char *`，而 Rust 中有那么多不同的分类。但是实际上，C 的 `char *` 在真正在使用的时候，还需要在不同的场景中学习那个场景中的知识，并在 `char *` 基础上继续小心翼翼地处理。而 Rust 中的这些字符串类型其实已

经包含了很多场景信息了，你直接用就行了，如果用得不对，在编译期间就会被指出。**类型化其实就是囊括更多规范信息的过程，让我们在编程的时候减少出错的几率。**

Rust 的错误处理也完全建立在标准化的类型系统上 (`Result<T, E>`)，这是一种与之前的主流语言完全不同的思路，因此开始学起来可能不太适应。但是其实一旦思路转换过来，会发现 Rust 的这套标准的错误处理思想更优美，更能降低我们的心智负担。

类型系统与传统的 OOP

trait 理论是一种平铺的理念，它与传统的 OOP 理念完全不同，甚至是完全对立的。你可以这样来理解，OOP 就像传统血缘社会，做什么都要看出身，而 trait 是现代法治社会，大家都平等。**在 Rust 中，所有 trait 都是平等的。**不管是 std 标准库中的 trait，还是你自定义的 trait，它们都是平等关系，不存在什么继承。

trait 可以类比为社会中的法律，给实体（自然人、法人等）以约束。法律约束你的同时，其实也给你界定了能做的事情，也就是你具有某些能力。因此类比过来，trait 体系实际是一种约束 + 能力体系。从社会的发展来看，Rust 这种 trait 架构下的类型体系，相比于 OOP，生产力可以更高。或者说 **Rust 的 trait 体系，实际更容易准确描述现实世界，因为现代世界主体就是按这种方式运行的。**

当然我们并不是说 OOP 就是错的。在某些场合，比如族谱模型、GUI 界面描述等，它确实更适合的抽象方法。因此你可以看到 Slint 这种 GUI 框架，在 Rust 中造出一个具有继承能力的描述性语言 slint。这就相当于把两种抽象方法的优势结合起来了。

从难度来讲，其实 trait 体系比完整的 OOP 体系更简单、知识点更少。开始时感觉难学，可能主要是思维需要转变。毕竟 OOP 思想占据了软件开发核心地位几十年了。

因此**学习 Rust 的两大核心就是所有权体系与 trait 体系。**它们就好像一根绳子的两头，只要你把两个头提起来了，那么其他东西很快就跟着起来了。并不是说其他东西不重要，它们都是构成 Rust 这个整体的重要组成部分。我们这里说的是学习方法，一定要把这两个主要的东西搞懂，其他的知识点相对比较独立，可以被这两样特性有效地串起来。

最后的寄语

你学习 Rust 的时候更重要的是学会转变思维，毕竟前面几十年的主流语言都采用了不同的思维模型。只要你花上一定时间适应了它，你就会发现 Rust 是如此优美、高效、富有创意。

Rust 是一位多面手，知识点很多。你可以先选择一个方向切入，使用 Rust 实现你最熟悉的或者最感兴趣的业务系统，产生价值回馈，然后再扩展到其他感兴趣的方面。就像我常说的，**学习和应用 Rust 没有天花板，天花板就是你的想象力。**

Rust 能应用在广泛的行业，我们这门课程只是刚起了个头，后面还有大量的知识点需要深入探索。学 Rust 就是学 CS 计算机科学，希望你与我一起持续探索，用 Rust 赋能各行各业。

听我说了这么多，最后我也想听听你的想法，这里我特别准备了一份 [🔗 结课问卷](#)，你可以花几分钟填写一下，希望能够听到你的“声音”。



唐刚

Rust 语言中文社区联合创始人

感谢一起走过的这段时间，非常想听听你对我和这门课程的反馈与建议。在2024年1月19日前提交问卷，将有机会获得



原创笔记本电脑内胆包

价值 **¥59**

或



极客时间课程阅码

价值 **¥99**

填写问卷 

全部留言 (5)

最新 精选



倪步烤

2024-01-05 来自浙江

好像少一讲，显示是35/36

编辑回复：30讲正文+3篇课后题+开篇词结束语测试题，一共36讲，你再看看，是不是显示已学35呀，还有一节未学



小虎子

2024-01-05 来自北京

我已经拿到结课证书了，谁有我快~



一带一路

2024-01-05 来自四川

太棒了，期待后续课程(●●●●●)



安

2024-01-05 来自广东

期待后续,我的老朋友



Forest

2024-01-05 来自四川

再来个系统性进阶实战课吧

